CONSTRAINT SATISFACTION PROBLEMS

Many problems require us to make a collection of decisions while respecting certain restrictions (constraints).

The following are some common examples:

1. Map Colouring Problem:
Given a map, assign colours to regions such that the neighbouring regions get different colours.

2. Exam Scheduling Problem:
Given a set of courses (where students may be enrolled in multiple courses) and a set of time slots, schedule the exams so that no student has two exams at the same time.

3. Sudoku:
Given a partially filled 9×9 Sudoku grid, assign numbers to the remaining cells so that each row, each column, and each 3×3 subgrid contains the numbers 1 through 9 exactly once.

A **Constraint Satisfaction Problem (CSP)** is defined by the following components:

Set of variables:
$\{X_1, X_2, \ldots, X_n\}$

Domains of the variables:
For each $1 \leq i \leq n$, the set D_i is the set of possible values of X_i.

Constraints:
Constraints specify allowable combinations of values.

An assignment of values to only some of the variables is called a *partial assignment*.

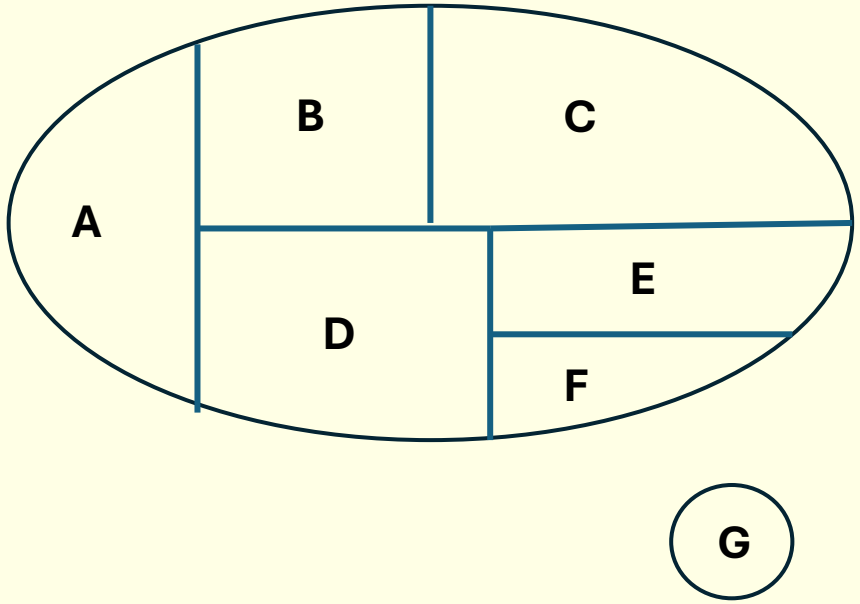
An assignment of values to only all the variables is called a *complete assignment*.

An assignment of values that does not violate any constraints is called a *consistent assignment*.

A solution to the CSP is a **complete** and **consistent** assignment.

Example 1 (Map colouring):

Given the following map, colour the regions using three colours such that the neighbouring regions get different colours.



Formulation as a CSP:

Variables: A, B, C, D, E, F, G

Domains:

$D_A = D_B = D_C = D_D = D_E = D_F = D_G = \{\text{red } (r), \text{green } (g), \text{blue } (b)\}$

Constraints:

$A \neq B, A \neq D, B \neq C, B \neq D, C \neq D, C \neq E, D \neq E, D \neq F, E \neq F$

The following is a solution to the CSP:

$A = r,\ B = g,\ C = r,\ D = b,\ E = g,\ F = r,\ G = r.$

Example 2 (Exam Scheduling):

Suppose the exams for the courses A, B, C, D have to be scheduled and the available time slots are S_1, S_2, S_3. Suppose that courses A, B have some common students, courses A, C have some common students, and courses B, D have some common students. Schedule the exams for the courses such that no student has two exams at the same time.

Formulation as a CSP:

Variables: A, B, C, D

Domains:
$D_A = D_B = D_C = D_D = \{S_1, S_2, S_3\}$

Constraints:
$A \neq B, A \neq C, B \neq D$

The following is a solution to the CSP:
$A = S_1,\ B = S_2,\ C = S_2,\ D = S_1.$

Example 3 (Sudoku):

Given a partially filled 9×9 Sudoku grid, assign numbers to the remaining cells so that each row, each column, and each 3×3 subgrid contains the numbers 1 through 9 exactly once.

Formulation as a CSP:

Variables: The 81 cells in the grid, denoted by $c_{11}, c_{12}, \ldots, c_{19}, c_{21}, \ldots, c_{29}, \ldots, c_{91}, c_{92}, \ldots, c_{99}$.

Domains:
For each variable, the domain is $\{1,2,3,4,5,6,7,8,9\}$.

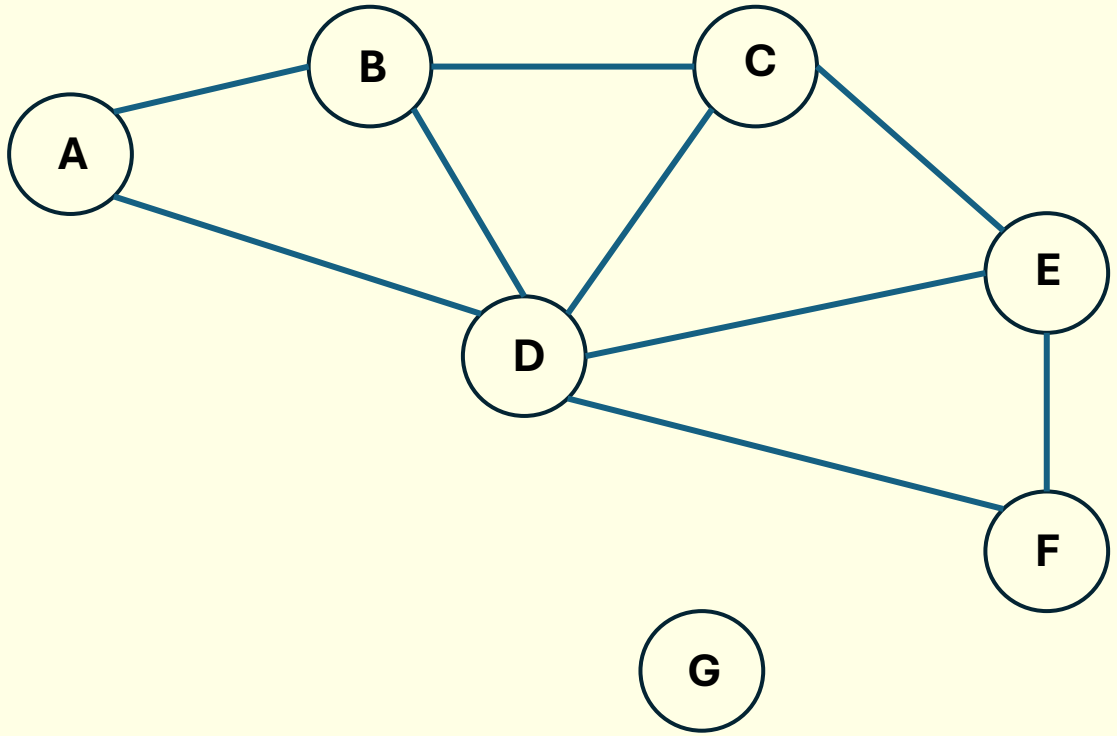
Constraints:
$c_{ij} = m_{ij}$ for each i, j such that c_{ij} is already filled with m_{ij};
$c_{ij} \neq c_{ik}$ for all $1 \leq i, j, k \leq 9, j \neq k$;
$c_{ij} \neq c_{kj}$ for all $1 \leq i, j, k \leq 9, i \neq k$;
$c_{ij} \neq c_{kl}$ for all i, j, k, l with $(i, j) \neq (k, l)$ and $(i, k), (j, l) \in A^2 \cup B^2 \cup C^2$,
where $A = \{1,2,3\}$, $B = \{4,5,6\}$, $C = \{7,8,9\}$.

Any correct solution to the sudoku is a solution to the CSP.

Constraint Graph

A CSP can be visualized through a *constraint graph*, where the nodes of the graph correspond to variables of the problem, and an edge connects any two variables that participate in a constraint.

For example, the constraint graph for the map colouring problem in Example 1 is the following:



Backtracking Search Algorithm for CSPs

function BACKTRACKING-SEARCH(*csp*) **returns** a solution or *failure*
  return BACKTRACK(*csp*, {})

function BACKTRACK(*csp*, *assignment*) **returns** a solution or *failure*
  if *assignment* is complete **then return** *assignment*
  var ← SELECT-UNASSIGNED-VARIABLE(*csp*, *assignment*)
  for each *value* **in** ORDER-DOMAIN-VALUES(*csp*, *var*, *assignment*) **do**
    if *value* is consistent with *assignment* **then**
      add {*var* = *value*} to *assignment*
      result ← BACKTRACK(*csp*, *assignment*)
      if *result* ≠ *failure* **then return** *result*
      remove {var = value} from assignment
  return *failure*

Consider the following CSP which corresponds to a map colouring problem:

Variables: A, B, C, D

Domains:

$D_A = \{\text{red } (r), \text{green } (g), \text{blue } (b)\}$
$D_B = \{\text{red } (r), \text{green } (g), \text{blue } (b)\}$
$D_C = \{\text{red } (r), \text{green } (g), \text{blue } (b)\}$
$D_D = \{\text{red } (r)\}$

Constraints:

$A \neq B, A \neq C, B \neq C, A \neq D$

Let the order of variables be fixed as A, B, C, D, and the order of the values to be assigned be fixed as red (r), green (g), blue (b).
The following is the trace of backtracking search algorithm for the given CSP:

$A = r$: Consistent. Assign value to next variable.
$A = r, B = r$: $A \neq B$ is violated. Backtrack to B.
$A = r, B = g$: Consistent. Assign value to next variable.
$A = r, B = g, C = r$: $A \neq C$ is violated. Backtrack to C.
$A = r, B = g, C = g$: $B \neq C$ is violated. Backtrack to C.
$A = r, B = g, C = b$: Consistent. Assign value to next variable.
$A = r, B = g, C = b, D = r$: $A \neq D$ is violated. Backtrack to D.
As there is no value left for D, backtrack to C.
As there is no value left for C, backtrack to B.
$A = r, B = b$: Consistent. Assign value to next variable.
$A = r, B = b, C = r$: $A \neq C$ is violated. Backtrack to C.
$A = r, B = b, C = g$: Consistent. Assign value to next variable.

(continued on next page)

$A = r, B = b, C = g, D = r : A \neq D$ is violated. Backtrack to D.

As there is no value left for D, backtrack to C.

$A = r, B = b, C = b : B \neq C$ is violated. Backtrack to C.

As there is no value left for C, backtrack to B.

As there is no value left for B, backtrack to A.

$A = g$: Consistent. Assign value to next variable.

$A = g, B = r$: Consistent. Assign value to next variable.

$A = g, B = r, C = r : B \neq C$ is violated. Backtrack to C.

$A = g, B = r, C = g : A \neq C$ is violated. Backtrack to C.

$A = g, B = r, C = b$: Consistent. Assign value to next variable.

$A = g, B = r, C = b, D = r$: Consistent and complete.

Solution by backtracking search: $A = g, B = r, C = b, D = r$.

Note how backtracking search performed many wasteful computations before realizing that the assignment for A has to be changed in the above example.

Backtracking search is a recursive depth-first-search performed on the search tree.

Note that in the above example, the value r can be removed from the domain of A before using backtracking search because there is no value in the domain of D that is consistent with $A = r$ (because of the constraint $A \neq D$); hence $A = r$ cannot be a part of any solution to the given CSP.

In general, such removals can be done by checking the *consistency of the arcs (edges)* of the constraint graph before using backtracking search.

Arc Consistency

A variable in a CSP is *arc consistent* if every value in its domain satisfies the variable's binary constraints.

More formally, a variable X_i is arc-consistent with respect to another variable X_j if for every value in the current domain D_i, there is some value in the domain D_j that satisfies the binary constraint on the arc (X_i, X_j).

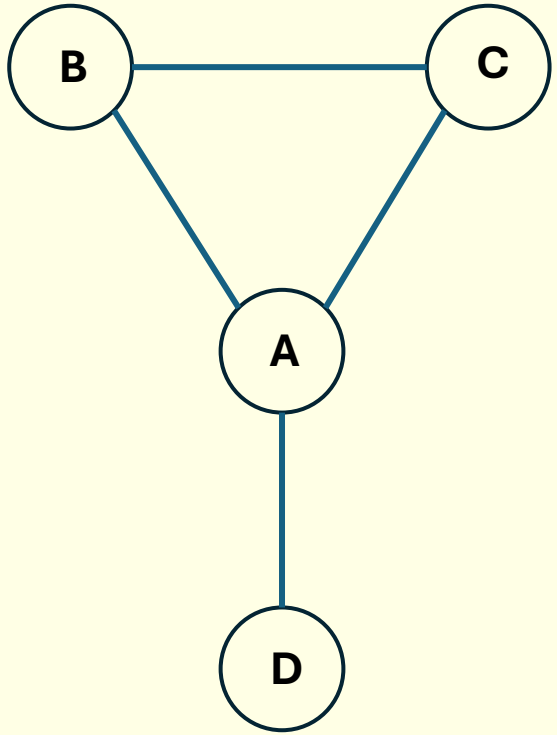
A CSP is arc consistent if every variable is arc consistent with every other variable.

Arc Consistency (AC-3) Algorithm

function AC-3(*csp*) **returns** *false* if an inconsistency is found and *true* otherwise
  queue ← a queue of arcs, initially all the arcs in *csp*
  while *queue* is not empty **do**
    (X_i, X_j) ← POP(*queue*)
    if REVISE(*csp*, X_i, X_j) **then**
      if size(D_i) = 0 **then return** *false*
      for each X_k **in** X_i.NEIGHBOURS $-$ $\{X_j\}$ **do**
        add (X_k, X_i) to *queue*
  return *true*

function REVISE(*csp*, X_i, X_j) **returns** *true* if and only if the domain of X_i is revised
  revised ← *false*
  for each *x* **in** D_i **do**
    if no value *y* in D_j allows (*x*, *y*) to satisfy the constraint between X_i and X_j **then**
      delete *x* from D_i
      revised ← *true*
  return *revised*

The following is the constraint graph for the map colouring example used to illustrate backtracking search:



Trace of AC-3 algorithm for the map colouring example:

$queue = \{(A,B),(B,A),(A,C),(C,A),(B,C),(C,B),(A,D),(D,A)\}$
(Note that **two ordered pairs** (directed arcs) are taken for every undirected edge in the constraint graph.)

POP($queue$) = (A,B):
REVISE(A,B) = *false* (after running the for loop)

POP($queue$) = (B,A):
REVISE(B,A) = *false* (after running the for loop)

POP($queue$) = (A,C):
REVISE(A,C) = *false* (after running the for loop)

POP($queue$) = (C,A):
REVISE(C,A) = *false* (after running the for loop)

(continued on next page)

POP($queue$) = (B, C):
REVISE(B, C) = *false* (after running the for loop)

POP($queue$) = (C, B):
REVISE(C, B) = *false* (after running the for loop)

POP($queue$) = (A, D):
REVISE(A, D):
for loop through values in D_A:
$x = r$: No value in D_D satisfies $A \neq D$; Delete r from D_A ; *revised* $\leftarrow$ *true*.
$x = g$: No deletion.
$x = b$: No deletion.
REVISE(A, D) = *true*
add (B, A) to *queue*
add (C, A) to *queue*

(continued on next page)

POP($queue$) = (D, A):
    REVISE(D, A) = *false* (after running the for loop)

POP($queue$) = (B, A):
    REVISE(B, A) = *false* (after running the for loop)

POP($queue$) = (C, A):
    REVISE(C, A) = *false* (after running the for loop)

queue is empty.
return *true*.

After applying AC-3 algorithm, the (reduced) CSP is the following:

Variables: A, B, C, D

Domains:

$D_A = \{\text{green } (g), \text{blue } (b)\}$
$D_B = \{\text{red } (r), \text{green } (g), \text{blue } (b)\}$
$D_C = \{\text{red } (r), \text{green } (g), \text{blue } (b)\}$
$D_D = \{\text{red } (r)\}$

Constraints:

$A \neq B, A \neq C, B \neq C, A \neq D$

The following is the trace of the backtracking search algorithm on the (reduced) CSP:

$A = g$: Consistent. Assign value to next variable.
$A = g, B = r$: Consistent. Assign value to next variable.
$A = g, B = r, C = r : B \neq C$ is violated. Backtrack to C.
$A = g, B = r, C = g : A \neq C$ is violated. Backtrack to C.
$A = g, B = r, C = b$: Consistent. Assign value to next variable.
$A = g, B = r, C = b, D = r$: Consistent and complete.

Solution by backtracking search: $A = g, B = r, C = b, D = r$.

Note that backtracking search now returned the solution in much fewer steps than before.

For a given CSP, we can first run AC-3 algorithm to eliminate the impossible values in the domains of the variables and then use backtracking search for the (reduced) CSP.

Note: It is also possible to incorporate consistency checks such as arc consistency in the backtracking search algorithm itself.

This is done using a function named INFERENCE, which imposes a consistency condition after the addition of each $\{var = value\}$ to *assignment*.

Backtracking Search Algorithm with INFERENCE function

function BACKTRACKING-SEARCH(*csp*) **returns** a solution or *failure*
  return BACKTRACK(*csp*, {})

function BACKTRACK(*csp*, *assignment*) **returns** a solution or *failure*
  if *assignment* is complete **then return** *assignment*
  var ← SELECT-UNASSIGNED-VARIABLE(*csp*, *assignment*)
  for each *value* **in** ORDER-DOMAIN-VALUES(*csp*, *var*, *assignment*) **do**
    if *value* is consistent with *assignment* **then**
      add {*var* = *value*} to *assignment*
      inferences ← INFERENCE(*csp*, *var*, *assignment*)
      if *inferences* ≠ *failure* **then**
        add *inferences* to *csp*
        result ← BACKTRACK(*csp*, *assignment*)
        if *result* ≠ *failure* **then return** *result*
        remove *inferences* from *csp*
      remove {*var* = *value*} from *assignment*
  return *failure*

The (plain) backtracking search algorithm given earlier can be seen as a special case of the backtracking search algorithm with INFERENCE function, with INFERENCE always returning an empty set.

Consider the following map colouring problem:

Variables: A, B, C, D, E, F, G

Domains:
$D_A = D_B = D_C = D_D = D_E = D_F = D_G = \{\text{red } (r), \text{green } (g), \text{blue } (b)\}$

Constraints:
$A \neq B, A \neq D, B \neq C, B \neq D, C \neq D, C \neq E, D \neq E, D \neq F, E \neq F$

Note that the AC-3 algorithm has **no effect** on this problem, although there are various inconsistent solutions (colourings) for this problem.

Consider the following triangle colouring problem:

Using two colours, colour the three vertices of a triangle in such a way that no two adjacent vertices get the same colour.

CSP formulation:

Variables: A, B, C
Domains: $D_A = D_B = D_C = \{\text{red}\ (r), \text{green}\ (g)\}$
Constraints: $A \neq B, B \neq C, A \neq C$

Note that this CSP has no solution; however, AC-3 algorithm has **no effect** on this problem.

Therefore, in general, AC-3 need not always reduce the domains of the variables.

Even when the CSP has no solution, AC-3 is not guaranteed to reduce the domains.

Arc consistency (which only looks at **two** variables at a time through a binary constraint) is blind to some inconsistencies such as in the above two examples.

The natural way out is to look at **three** variables at once. This is what is known as *path consistency*.

The set of two variables $\{A, B\}$ is *path-consistent* with respect to a (third) variable C if for every value pair $(a, b) \in D_A \times D_B$ that is consistent with the A-B constraint, there exists $c \in D_C$ consistent with both A-C and B-C constraints.

- **Node consistency (1-consistency):** A variable X_i is *node consistent* if every value in D_i satisfies all unary constraints on X_i. A CSP is *node consistent* if every variable is node consistent.

- **Arc consistency (2-consistency):** A variable X_i is *arc consistent* with respect to X_j if for every consistent value in $a \in D_i$, there exists $b \in D_j$ satisfying the binary constraint on (X_i, X_j). A CSP is *arc consistent* if every variable is arc consistent with respect to every other variable.

- **Path consistency (3-consistency):** A pair of variables $\{X_i, X_j\}$ is *path consistent* with respect to a variable X_m if for every value pair $(a, b) \in D_i \times D_j$ that is consistent with the X_i-X_j constraint, there exists $c \in D_m$ satisfying the binary constraints on (X_i, X_m) and (X_j, X_m). A CSP is *path consistent* if every pair of variables is path consistent with respect to every other variable.

k-consistency

A set of $k-1$ variables $\{X_1, X_2, \ldots, X_{k-1}\}$ is *k-consistent* with respect to a variable X_k if for every $(a_1, a_2, \ldots, a_{k-1}) \in D_1 \times D_2 \times \cdots \times D_{k-1}$ that is consistent with the binary constraints involving $X_1, X_2, \ldots, X_{k-1}$, there exists $a_k \in D_k$ satisfying the binary constraints on (X_i, X_k) for all $1 \leq i \leq k-1$.

A CSP is *k-consistent* if every set of $k-1$ variables is k-consistent with respect to every other variable.

Strong *k*-consistency

A CSP is *strongly k-consistent* if it is k-consistent, $(k-1)$-consistent, $(k-2)$-consistent, …, 2-consistent and 1-consistent.

Suppose we have a CSP with n variables and make it strongly n-consistent (that is, strongly k-consistent for $k = n$).

We can then find a solution to the CSP as follows:

Step-1: Choose a consistent value for X_1. (This is possible because the CSP is 1-consistent).
Step-2: Choose a consistent value for X_2. (This is possible because the CSP is 2-consistent).

⋮

⋮

Step-n: Choose a consistent value for X_n. (This is possible because the CSP is n-consistent).

Worst-case time complexity to find a solution:
$1 + d + 2d + 3d + \cdots + (n-1)d = O(n^2 d)$.

Does this mean that CSPs can be solved in polynomial-time?

NO. Enforcing n-consistency itself must take exponential time.

What does enforcing k-consistency mean for some k?

For every relevant subset of $k-1$ variables, we need to examine which of the up to d^{k-1} possible joint assignments to that subset can be extended to every other variable.
When $k = n$, this means potentially reasoning about d^{n-1} possible assignments of values, which is exponential in n.

Unless P = NP, no polynomial algorithm for enforcing n-consistency can exist.
(**P = NP** is a well-known unsolved problem in computer science.)

The key takeaway for solving CSPs

- Do not aim for full consistency.
- Aim for cheap, partial consistency as a filter before/during search.

In practice, node and arc consistency (AC-3) (and sometimes path consistency) are used - not to *solve* the CSP, but to *prune domains* before using the backtracking search on the (reduced) CSP.